

МИНОБРНАУКИ РОССИИ
Федеральное государственное автономное
образовательное учреждение высшего образования
«Пермский государственный национальный
исследовательский университет»

Институт компьютерных наук и технологий

ОТЧЁТ
по индивидуальной работе №2
по дисциплине «Языки программирования»
Вариант 5

Работу выполнил студент группы ИТ/О
ИТ-7-2025 НБ 1 курса
Гороховская А.А.
«19» июня 2026 г.

Работу проверила
Ширина А.В.
«19» июня 2026 г

Пермь 2026

СОДЕРЖАНИЕ

Задача №1	3
Постановка задачи.....	3
Алгоритм решения	3
Шаг 1. Проверка входных данных и заполнение структуры	4
Шаг 2. Перебор всех возможных «начал» числа А.....	4
Шаг 3. Перебор длины числа А	4
Шаг 4. Перебор длины числа С и вычисление длины В	5
Шаг 5. Эвристика по последним цифрам (главная оптимизация).....	5
Шаг 6. Полная математическая проверка.....	6
Шаг 7. Завершение работы.....	6
Краткая схема алгоритма (псевдокод)	6
Классы	7
Функции.....	8
Тестирование	8
Сводная таблица тестов.....	9
Задача №2.....	12
Постановка задачи.....	12
Алгоритм решения	13
Шаг 1. Проверка входных данных и построение дерева.....	13
Шаг 2. Поиск максимальной глубины дерева (DFS)	14
Шаг 3. Подсчёт пустых гнёзд	14
Шаг 4. Применение формулы	14
Математическое обоснование формулы	15
Классы	17
Функции	17
Тестирование	18
Анализ результатов тестирования.....	21

Задача №1

Постановка задачи

Числовое кольцо. В кольце записаны N чисел, составляющих по часовой стрелке 3 числа: два слагаемых и сумму. Требуется написать программу, которая по строке чисел, считая ее кольцом, находит какое-нибудь решение в виде $A+B=C$. Все цифры числа должны входить в числа только один раз и в порядке следования в кольце. Цифр в кольце не более 1000.

Формат входных данных: Входной файл содержит 1 строку, в которой без пробелов перечислены цифры кольца.

Формат выходных данных: В выходной файл выводится тождество в виде $\langle \text{слагаемое1} \rangle + \langle \text{слагаемое2} \rangle = \langle \text{сумма} \rangle$ без пробелов внутри или слово «No», если решения не существует.

Пример 1:

(входные данные)

01902021

(выходные данные)

190+20=210

Пример 2:

(входные данные)

111111

(выходные данные)

No

Алгоритм решения

Программа решает задачу методом **полного перебора с оптимизацией**. Идея простая: перебираем все возможные способы разрезать кольцо на три куса (числа A , B и C) и проверяем, выполняется ли равенство $A + B = C$. Но чтобы не делать миллионы лишних проверок, добавлена «умная» эвристика, которая отбрасывает заведомо плохие варианты ещё до того, как мы начнём считать.

Шаг 1. Проверка входных данных и заполнение структуры

Когда программа получает строку (например, "01902021"), она первым делом проверяет: Строка не пустая?

В строке только цифры (нет букв, пробелов, знаков + и т.д.)?

Длина строки $N \geq 3$? (Меньше трёх цифр — невозможно сделать три числа.)

Если что-то из этого не выполняется — программа сразу говорит «No» или выдаёт ошибку.

Если всё ок, каждая цифра строки по очереди добавляется в **циклический двусвязный список**. Это наша «виртуальная модель кольца»:

Первый узел содержит '0', второй '1', третий '9' и так далее.

Последний узел «замыкается» на первый: у него `next` указывает на голову, а у головы `prev` указывает на хвост.

Благодаря этому, когда мы доходим до конца кольца, мы автоматически «заворачиваем» обратно в начало — не нужно делать никаких хитрых вычислений с индексами.

Шаг 2. Перебор всех возможных «начал» числа A

Кольцо не имеет фиксированного начала — это главное, что усложняет задачу. Поэтому мы должны попробовать **каждую цифру кольца** в роли первой цифры числа A.

Для этого мы используем метод `get_all_start_nodes()` — он по очереди возвращает каждый узел списка. Если в кольце 8 цифр, будет 8 итераций внешнего цикла.

Шаг 3. Перебор длины числа A

Для фиксированной начальной позиции перебираем, сколько цифр может занимать число A. Длина `len_a` может быть от 1 до $N-2$ (минус два, потому что нужно оставить хотя бы по одной цифре на числа B и C).

Например, для строки "01902021" ($N=8$) и начала во втором узле (цифра '1'):

`len_a = 1` → A = "1"

`len_a = 2` → A = "19"

`len_a = 3` → A = "190"

... и так далее до `len_a = 6`.

После того как мы взяли `len_a` цифр для A, мы запоминаем узел, с которого начинается число B (это просто `len_a` шагов вперёд от начала A).

Шаг 4. Перебор длины числа C и вычисление длины B

Дальше перебираем длину числа C — `len_c` (от 1 до $N - \text{len}_a - 1$).

A длину числа B мы **не перебираем** — она вычисляется автоматически:

$$\text{len}_b = N - \text{len}_a - \text{len}_c$$

Это ключевой момент: мы гарантируем, что все три числа вместе используют ровно N цифр кольца, без пропусков и без повторов.

Шаг 5. Эвристика по последним цифрам (главная оптимизация)

Здесь начинается самое интересное. Прежде чем делать «тяжёлую» проверку (извлекать полные строки, переводить их в числа, складывать), мы делаем **быструю проверку по последним цифрам**.

Вспомним школьную математику: если $A + B = C$, то последняя цифра числа C равна (последняя цифра A + последняя цифра B) mod 10.

Например:

A = 190 (последняя цифра 0)

B = 20 (последняя цифра 0)

C = 210 (последняя цифра 0)

Проверка: $(0 + 0) \bmod 10 = 0$ ✓ — совпадает, идём дальше.

A если бы было A = 191, B = 20, C = 210:

$(1 + 0) \bmod 10 = 1$, а последняя цифра C = 0.

$1 \neq 0$ — значит, такой вариант **точно не подходит**, и мы его пропускаем, даже не считая $191 + 20$.

Почему это круто: из 10 возможных комбинаций последних цифр только одна даёт нужную последнюю цифру C. То есть мы **отбрасываем около 90% вариантов** ещё до того, как начнём с ними полноценно работать. Это превращает фактическую сложность из $O(N^3)$ примерно в $O(N^2)$.

В коде это выглядит так:

Берём последнюю цифру A: `last_a = int(str_a[-1])`

Идём по связному списку от начала В на $\text{len_b} - 1$ шагов вперёд — находим последнюю цифру В.

Идём от начала С на $\text{len_c} - 1$ шагов вперёд — находим последнюю цифру С.

Проверяем: $(\text{last_a} + \text{last_b}) \% 10 == \text{last_c}$. Если нет — continue, переходим к следующему варианту.

Шаг 6. Полная математическая проверка

Если эвристика прошла (последние цифры совпали), значит, вариант **возможно** правильный. Теперь делаем полную проверку:

Извлекаем из связного списка полные строки В и С (методом `get_substring`).

Переводим все три строки в целые числа: `val_a, val_b, val_c`.

Проверяем: `val_a + val_b == val_c`.

Если равенство верно — **мы нашли решение!** Формируем строку вида "A+B=C" и возвращаем её.

Шаг 7. Завершение работы

Если мы перебрали все начальные позиции, все длины А, все длины С, и ни один вариант не подошёл — значит, решения не существует. Возвращаем "No".

Краткая схема алгоритма (псевдокод)

для каждого узла `start_node` в кольце: # перебор начала А

для `len_a` от 1 до `N-2`: # перебор длины А

`A = подстрока(start_node, len_a)`

`node_b = узел после А`

для `len_c` от 1 до `N - len_a - 1`: # перебор длины С

`len_b = N - len_a - len_c`

Быстрая проверка по последним цифрам

если $(\text{последняя_цифра}(A) + \text{последняя_цифра}(B)) \bmod 10 \neq \text{последняя_цифра}(C)$:

пропустить этот вариант

Полная проверка

B = подстрока(node_b, len_b)

C = подстрока(после_B, len_c)

если $\text{int}(A) + \text{int}(B) == \text{int}(C)$:

вернуть "A+B=C"

вернуть "No"

Такой алгоритм работаем правильно по нескольким причинам:

Полнота перебора. Мы рассматриваем все возможные начала кольца (N штук) и все возможные разбиения на три части. Ни один вариант не пропущен.

Корректность эвристики. Проверка по модулю 10 — это математический факт: последняя цифра суммы всегда определяется только последними цифрами слагаемых. Мы не можем отбросить верный вариант.

Циклический список. Благодаря тому, что список замкнут, нам не нужно «склеивать» строку с самой собой (как делают, когда работают с обычной строкой). Обход получается естественным и быстрым.

Ранний выход. Как только нашли первое подходящее решение — сразу возвращаем его. По условию задачи достаточно найти любое решение, поэтому искать дальше нет смысла.

Классы

- Node: хранит символ (data) и ссылки на следующий (next) и предыдущий (prev) узлы.
- CircularLinkedList: хранит указатель на голову (head) и размер списка (size). Метод append корректно замыкает цепочку, а метод get_substring извлекает последовательность символов, начиная с произвольного узла.

Функции

Программа разделена на логические модули согласно принципу единственной ответственности:

Базовые операции со структурой данных:

- `append(self, data)` — добавляет новый узел в конец циклического списка и обновляет связи хвоста с головой.
- `get_substring(self, start_node, length)` — проходит по `length` узлов, начиная с `start_node`, собирая символы в строку. Критически важна для извлечения чисел A, B и C.
- `get_all_start_nodes(self)` — генератор, возвращающий все узлы списка по очереди. Позволяет перебирать все возможные точки начала числа A без создания дополнительных копий данных.

Логика решения задачи:

- `solve(self)` — основной метод поиска тождества. Перебирает все комбинации длин чисел A, B и C. Перед полным вычислением применяет эвристическую оптимизацию $(last_a + last_b) \% 10 == last_c$, что отсекает ~90% неподходящих вариантов за $O(1)$. При нахождении первого валидного решения немедленно возвращает результат.

Пользовательский интерфейс и ввод-вывод:

- `main_menu()` — реализует бесконечный цикл меню с четырьмя пунктами. Обрабатывает неверный выбор пункта.
- `get_valid_string_input(prompt)` — запрашивает ввод до тех пор, пока пользователь не введёт непустую строку, состоящую исключительно из цифр.
- `solve_from_file()` — читает входной файл, решает задачу и записывает результат в выходной файл. Содержит полную обработку исключений (`FileNotFoundError`, `ValueError`).
- `clear_screen()` — кроссплатформенная очистка консоли через `os.system`

Тестирование

Для проверки корректности работы программы был написан отдельный тестовый скрипт `test5.py`, который запускает программу на 30 различных сценариях и

автоматически проверяет результаты. Тесты делятся на несколько блоков: базовые сценарии из задания, граничные случаи, проверка ведущих нулей, циклические свойства, разная длина, случаи без решения, проверка самой структуры данных и тест производительности.

Программа проверяет не только совпадение с ожидаемой строкой, но и математическую корректность ответа (что $A + B$ действительно равно C) и полноту покрытия кольца (что сумма длин трёх чисел равна длине исходной строки).

Сводная таблица тестов

Блок 1. Базовые сценарии

№	Название	Что вводится	Что ожидается	Что выведено	Результат
B1	Пример 1 из задания (сдвиг)	01902021	$19+2=21$	$19+2=21$	Пройден
B2	Пример 2 из задания	111111	No	No	Пройден
B3	Минимальная длина $N=3$	123	$1+2=3$	$1+2=3$	Пройден

Блок 2. Граничные случаи

№	Название	Что вводится	Что ожидается	Что выведено	Результат
G1	Все нули	0	$0+0=0$	$0+0=0$	Пройден
G2	Слишком короткая строка	12	No	No	Пройден
G3	Ведущий ноль в результате	1001	$1+0=1$	$1+0=1$	Пройден
G4	Все девятки	9999	No	No	Пройден
G5	Два нуля и единица	1	No	No	Пройден
G6	Чередование 1 и 0	1010	$0+1=1$	$0+1=1$	Пройден

Блок 3. Ведущие нули

№	Название	Что вводится	Что ожидается	Что выведено	Результат
LZ1	Ведущие нули в начале	00123	$1+2=3$	$1+2=3$	Пройден
LZ2	Нули внутри чисел	100200300	$100+200=300$	$100+200=300$	Пройден
Z3	Нули между числами	010203	$1+2=3$	$1+2=3$	Пройден

Блок 4. Циклические свойства

№	Название	Что вводится	Что ожидается	Что выведено	Результат
C1	Сдвиг примера 1	210019020	19+2=21	19+2=21	Пройден
C2	Другой сдвиг	20210019	19+2=21	19+2=21	Пройден
C3	Сдвиг 1+2=3	312	1+2=3	1+2=3	Пройден

Блок 5. Различная длина

№	Название	Что вводится	Что ожидается	Что выведено	Результат
D1	Длина 6 (нет решения)	123456	No	No	Пройден
D2	Длина 6 с повторами	112233	11+22=33	11+22=33	Пройден
D3	Длина 9 (нет решения)	123456789	No	No	Пройден
D4	Длинная строка из одинаковых цифр	555...5 (100 шт.)	No	No	Пройден

Блок 6. Случай без решения

№	Название	Что вводится	Что ожидается	Что выведено	Результат
N1	Три одинаковые цифры	555	No	No	Пройден
N2	Убывающая последовательность	987	No	No	Пройден
N3	Нечётные числа	135	No	No	Пройден
N4	Чётные числа	2468	No	No	Пройден

Блок 7. Проверка структуры данных CircularLinkedList

№	Название	Что вводится	Что ожидается	Что выведено	Результат
S1	Создание и заполнение списка	Добавление «1», «2», «3»	`size == 3`, `head is not None`	Структура создана	Пройден
S2	Проверка цикличности	tail.next == head` и `head.prev == tail`	Истина	Цикличность подтверждена	Пройден
S3	Извлечение подстроки	Из «01902021» со 2-го узла, длина 3	190	Подстрока извлечена	Пройден

Блок 8. Валидация входных данных

№	Название	Что вводится	Что ожидается	Что выведено	Результат
V1	Пустая строка	""(пустая строка)	ValueError — строка неотклонена может быть пустой	Пустая строка	Пройден
V2	Буквы в строке	12a34	ValueError — строка должна содержать только цифры	Буквы отклонены	Пройден
V3	Спецсимволы	12+34	ValueError — строка должна содержать только цифры	Спецсимволы отклонены	Пройден

Блок 9. Тест производительности

№	Название	Что вводится	Что ожидается	Что выведено	Результат
P1	Строка из 1000 цифр	"1"+ 499 нулей + "1" + 499 нулей	No	Найдено решение	Провален

Анализ «провала»:

Данный результат **не является ошибкой алгоритма**. После проверки выяснилось, что в такой длинной строке действительно существует решение — например, разбиение, в котором одно из чисел начинается с единицы, а остальные цифры — нули, что даёт верное равенство. Программа отработала корректно:

не «зависла» на большом объёме данных;

нашла математически верное решение;

проверила полноту покрытия кольца (сумма длин A, B и C равна 1000).

«Провал» связан с некорректно составленным ожидаемым результатом в самом тесте, а не с багом в программе.

Задача №2

Постановка задачи

«Почтальон» Мудрый тритон работает почтальоном в Лукоморье. В перечень его обязанностей входит доставка корреспонденции птицам, разместившим свои гнезда на ветвях дуба (постепенно вытеснив оттуда русалку). Птицы выют свои гнезда там, где ветка дерева разделяется на несколько более тонких веток. Лукоморье является районом с развитой инфраструктурой, поэтому все разветвления на дубе заняты гнездами птиц. За время работы тритон не только запомнил, где кто живет, но и уяснил, что если его путь проходит через чье-либо гнездо, и его обитатели не получают при этом корреспонденции, ему приходится извиняться за доставленные неудобства. Естественно, интеллигентный тритон стремится как можно меньше беспокоить обитателей дуба, поэтому он заранее планирует свой маршрут так, чтобы приносить минимальное число извинений.

Напишите программу, которая по описанию мест обитания птиц и списку корреспонденции определяет, сколько раз придется извиняться мудрому тритону.

Каждое гнездо дуба обозначено уникальным номером, что соответствует номеру квартиры, который мы иногда указываем, отправляя письма родным и близким по обычной почте. Гнездо, расположенное на первой развилке ствола имеет номер один. Тритон может только ползти по веткам или стволу дуба, прыжки и перелеты полностью исключены. Никакие ветки не соприкасаются и не пересекаются.

Формат входного файла: в первой строке входного файла находится число M – количество гнезд на дубе ($0 < M < 1000$). В следующих M строках находится информация о размещении гнезд и количестве писем для каждого из возможных адресатов. Первые два числа n_i и l_i – количество соседей гнезда с номером i и количество направленных по этому адресу писем соответственно. Далее в строке приводится n_i чисел – номера гнезд, являющихся соседними с гнездом с номером i .

Формат выходного файла: в единственной строке выходного файла должно содержаться единственное число – минимальное количество извинений,

которые придется приносить мудрому тритону, для того чтобы доставить все письма.

Пример:

(входной файл)

5

3 2 2 5 4

1 1 1

1 1 4

2 2 3 1

1 3 1

(выходной файл)

2

Алгоритм решения

Программа решает задачу не методом симуляции маршрута (это было бы очень сложно), а через **математическую формулу**, которая выводится из свойств дерева. Идея в том, что мы можем посчитать количество извинений, даже не зная самого маршрута — достаточно знать три числа: количество гнёзд, глубину дерева и количество пустых гнёзд.

Шаг 1. Проверка входных данных и построение дерева

Когда программа читает входной файл, она первым делом проверяет:

- Первая строка — число M в диапазоне от 1 до 999?
- Количество строк соответствует заявленному M ?
- В каждой строке корректные числа и правильное количество соседей?

Если что-то не так — программа сразу выбрасывает `ValueError` с понятным сообщением. Пользователь видит, где именно ошибка.

Если всё ок, программа строит **дерево** — специальную структуру данных, где:

- Каждое гнездо — это объект класса `Nest` с полями `number` (номер), `letters` (количество писем) и `neighbors` (список номеров соседей).
- Все гнёзда хранятся в словаре `nests` внутри класса `Tree` — это позволяет быстро находить любое гнездо по номеру за $O(1)$.
- Корень дерева — всегда гнездо с номером 1 (по условию, это первое разветвление ствола).

Почему именно дерево? По условию задачи ветки дуба не пересекаются, гнёзда расположены в разветвлениях, и из любого гнезда можно добраться до любого другого единственным путём. Это классическое определение дерева в теории графов: связный граф без циклов.

Шаг 2. Поиск максимальной глубины дерева (DFS)

Это ключевой параметр для формулы. **Глубина** — это расстояние от корня (гнезда №1) до самого далёкого листа (гнезда без потомков).

Для поиска используется **рекурсивный обход в глубину (DFS)**. Алгоритм работает так:

1. Начинаем с корня, его глубина = 0.
2. Для каждого соседа текущего узла (кроме родителя, чтобы не ходить назад) рекурсивно вызываем функцию с увеличенной на 1 глубиной.
3. Возвращаем максимальное значение глубины, которое встретилось.

Почему рекурсия, а не итеративный стек

Для $N < 1000$ глубина рекурсии не превышает 1000, а мы подняли лимит через `sys.setrecursionlimit(2000)`. Рекурсивный код читается понятнее, чем итеративный со стеком — это важно для защиты.

Шаг 3. Подсчёт пустых гнёзд

Проходим по всем гнёздам дерева и считаем, у скольких из них `letters == 0`. Это значение Z — количество гнёзд, для которых нет корреспонденции.

Почему это важно? Когда тритон заходит в гнездо **впервые**, он извиняется только если там нет писем (не за что здороваться). Если письма есть — он просто отдаёт их и идёт дальше без извинений.

Шаг 4. Применение формулы

Финальный шаг — подставляем три числа в формулу:

$$N - 1 - \text{max_depth} + Z$$

где:

- N — количество гнёзд (размер дерева)
- max_depth — максимальная глубина от корня
- Z — количество пустых гнёзд

Математическое обоснование формулы

Чтобы обойти всё дерево и **вернуться в корень**, тритону пришлось бы пройти по каждому ребру ровно 2 раза (туда и обратно). В дереве из N узлов всегда $N-1$ ребро, значит, полный обход $= 2 \times (N - 1)$ переходов.

Но тритону **не нужно возвращаться в корень!** Он может закончить маршрут в самом глубоком гнезде. Это экономит ему max_depth переходов — ровно столько, сколько рёбер на пути от корня до этого листа.

Итого переходов: $2(N - 1) - \text{max_depth}$

1. Сколько раз тритон заходит в гнёзда?

Каждый переход — это заход в очередное гнездо. Всего заходов = количество переходов + 1 (первый заход в корень в самом начале):

$$\text{всего_заходов} = 2(N - 1) - \text{max_depth} + 1 = 2N - 2 - \text{max_depth} + 1 = 2N - 1 - \text{max_depth}$$

Когда тритон заходит в гнездо **впервые**, он:

- Если там есть письма ($l > 0$) — отдаёт их, извиняться не нужно.
- Если там нет писем ($l = 0$) — извиняется за беспокойство.

Всего «первичных» заходов в гнёзда ровно N (каждое гнездо посещается впервые ровно один раз). Из них Z — это заходы в пустые гнёзда, которые требуют извинений.

Значит, **первичные извинения** = Z .

Все остальные заходы (повторные посещения гнёзд) — это возвраты тритона по уже пройденным веткам. Каждый такой заход требует извинений, потому что тритон беспокоит жителей, не неся им корреспонденции.

Количество повторных заходов:

$$\text{повторные} = \text{всего_заходов} - N = (2N - 1 - \text{max_depth}) - N = N - 1 - \text{max_depth}$$

Считаем итоговое количество извинений

извинения = первичные + повторные = $Z + (N - 1 - \text{max_depth})$

Формула: $N - 1 - \text{max_depth} + Z$ подходит

Краткая схема алгоритма (псевдокод)

1. Прочитать M, проверить корректность
2. Для каждого гнезда i от 1 до M:
 - прочитать n_i, l_i и список соседей
 - создать объект Nest(i, l_i)
 - сохранить в словарь nests
3. N = количество гнёзд (должно быть равно M)
4. Найти максимальную глубину от корня (гнездо 1):
 - функция dfs_depth(узел, родитель, глубина):
 - max_d = глубина
 - для каждого соседа узла:
 - если сосед != родитель:
 - d = dfs_depth(сосед, узел, глубина + 1)
 - если d > max_d:
 - max_d = d
 - вернуть max_d
 - max_depth = dfs_depth(корень, -1, 0)
5. Посчитать количество пустых гнёзд:
 - Z = 0
 - для каждого гнезда в nests:
 - если letters == 0:
 - Z = Z + 1
6. Ответ = $N - 1 - \text{max_depth} + Z$
 - вернуть ответ

Классы

- Nest: хранит номер гнезда (number), количество писем (letters) и список номеров соседних гнёзд (neighbors).

- Tree: реализует дерево на основе словаря смежности. Хранит коллекцию узлов (nests) и номер корня. Предоставляет методы для построения структуры и вычисления метрик (глубина, счетчик пустых узлов).

- PostmanSolver: инкапсулирует логику решения задачи. Принимает объект Tree и вычисляет итоговое значение по формуле, использующей параметры дерева.

Функции

Программа разделена на логические модули согласно принципу единственной ответственности:

Базовые операции со структурой данных:

- add_nest(self, number, letters) — создает и добавляет новый узел Nest в словарь дерева. Критически важна для инициализации структуры из входных данных.

- set_neighbors(self, number, neighbors) — устанавливает связи между узлами, заполняя список смежности. Позволяет задать топологию дерева.

- _dfs_depth(self, current, parent, depth) — рекурсивный обход в глубину (DFS). Находит максимальную глубину дерева от корня, игнорируя обратный переход к родителю.

- count_empty_nests(self) — линейный проход по всем узлам для подсчета гнёзд без писем (где letters == 0).

Логика решения задачи:

- solve(self) — основной метод расчета. Вычисляет результат по формуле $N - 1 - \max_depth + \text{empty_count}$, объединяя метрики размера, глубины и заполненности дерева.

- build_tree_from_lines(lines) — парсер входных данных. Преобразует текстовые строки в объект Tree, выполняя строгую валидацию формата (проверка количества соседей, диапазонов чисел).

Пользовательский интерфейс и ввод-вывод:

- main_menu() — реализует бесконечный цикл меню с тремя пунктами. Обрабатывает неверный выбор пользователя.

- solve_manual() — организует защищенный ручной ввод данных с клавиатуры с пошаговой проверкой корректности каждой строки.

- solve_from_file() — читает входной файл, решает задачу и записывает результат в выходной файл. Содержит полную обработку исключений (FileNotFoundError, ValueError).

•clear_screen() — кроссплатформенная очистка консоли через os.system.

Тестирование

1. Базовые сценарии

Проверяется корректность работы алгоритма на типовых входных данных различной структуры.

№	Название	Что вводится	Что ожидается	Что выведено	Результат
1	Пример из условия задачи	5 3 2 2 5 4 1 1 1 1 1 4 2 2 3 1 1 3 1	2	2	Пройден
2	Дерево-линия (1-2-3-4-5), все с письмами	5 1 1 2 2 1 1 3 2 1 2 4 2 1 3 5 1 1 4	0	0	Пройден
3	Дерево-звезда (1 в центре, 2-3-4-5 листья)	5 4 1 2 3 4 5 1 1 1 1 1 1 1 1 1 1 1 1	3	3	Пройден
4	Полное бинарное дерево (7 узлов)	7 2 1 2 3 3 1 1 4 5 3 1 1 6 7 1 1 2 1 1 2 1 1 3	4	4	Пройден

		1 1 3			
5	Простейшее дерево из 2 узлов	2 1 5 2 1 3 1	0	0	Пройден

2. Граничные случаи

Проверяется работа программы на крайних значениях входных параметров: минимальное/максимальное количество узлов, пустые и переполненные данные.

№	Название	Что вводится	Что ожидается	Что выведено	Результат
6	Одно гнездо (корень) с письмом	1 0 1	0	0	Пройден
7	Одно гнездо без писем	1 0 0	1	1	Пройден
8	Все гнёзда без писем (линия из 3)	3 1 0 2 2 0 1 3 1 0 2	3	3	Пройден
9	Длинна я линия из 10 узлов	10 1 1 2 2 1 1 3 ... (всего 11 строк)	0	0	Пройден
10	Большо е количество писем (не влияет на ответ)	3 1 100 2 2 999 1 3 1 500 2	0	0	Пройден

3. Обработка ошибок (некорректные данные)

Проверяется корректная реакция программы на невалидный пользовательский ввод: буквы вместо чисел, выход за границы диапазона, неполные данные.

№	Название	Что вводится	Что ожидается	Что выведено	Результат
11	Пустые входные данные	(пустой список)	Исключение 'ValueError'	ValueError	Пройден
12	Первая строка — не число	Abc 1 1 2	Исключение 'ValueError'	ValueError	Пройден
13	M = 0 (вне диапазона)	0	Исключение 'ValueError'	ValueError	Пройден
14	M = 1000 (вне диапазона)	1000	Исключение 'ValueError'	ValueError	Пройден
15	Недостаточно строк (заявлено 3, есть 1)	3 1 1 2	Исключение 'ValueError'	ValueError	Пройден
16	Неверное число соседей (заявлено 2, указан 1)	2 2 1 2 1 1 1	Исключение 'ValueError'	ValueError	Пройден
17	Буквы вместо чисел в данных	2 1 abc 2 1 1 1	Исключение 'ValueError'	ValueError	Пройден

4. Работа с файлами

Проверяется корректность операций чтения входных файлов и обработки файловых ошибок.

№	Название	Что вводится	Что ожидается	Что выведено	Результат
18	Чтение корректного файла	Файл tmp236qov7l.txt (корректные данные)	2	2	Пройден
19	Чтение несуществующего файла	Файл nonexistent_file	Исключение FileNotFoundError	FileNotFoundError	Пройден

	щего файла	_12345.txt	rror		
20	Чтение пустого файла	Файл tmp8bi3ftfr.txt (пустой)	Исключение ValueError	ValueError	Пройден
21	Файл с некорректным содержимым	Файл tmpd3j1d69l.txt (содержит `not a number`)	Исключение ValueError	ValueError	Пройден

Анализ результатов тестирования

Корректность алгоритма. Все базовые сценарии (пример из условия, дерево-линия, звезда, бинарное дерево) дали ожидаемые результаты. Это подтверждает правильность формулы $N - 1 = D + Z$, где N — количество узлов, D — максимальная глубина, Z — количество пустых гнёзд.

Граничные случаи. Программа корректно обрабатывает вырожденные случаи: дерево из одного узла (с письмом и без), длинные линии (глубина рекурсии до 10 не вызывает `RecursionError` благодаря `sys.setrecursionlimit(2000)`), большие значения количества писем.

Устойчивость к некорректным данным. Все 7 тестов на обработку ошибок пройдены: программа не «падает» при невалидном вводе, а корректно возбуждает `ValueError` с информативным сообщением.

Файловые операции. Чтение корректных файлов работает, все типы файловых ошибок (`FileNotFoundError`, пустой файл, битый файл) корректно перехватываются и обрабатываются.

Вывод. Программа прошла **комплексное тестирование** и готова к демонстрации на защите. Все заявленные функции работают корректно во всех предусмотренных режимах.